

TECHNICAL ARCHITECTURE — B2B FITOUT PLATFORM

[Database types](#) · [REST API](#) · [Firebase SDK](#) · [Matching Pipeline](#)

CLIENT LAYER — Browser

[B2B_Fitout_Dashboard.html](#)

Vanilla JS · Firebase SDK v10
JSZip CDN · global scope
onSnapshot() realtime

act-generator.js

OOXML XML hand-rolled
JSZip → Blob → download()
zero server — client only

CLIENT LAYER — Catalog Matcher (React 18 · Vite · localhost:5173)

Catalog Matcher UI

React 18 · Vite · App.jsx
fetch() → REST /api/*
styles.css design tokens

Photo QA UI (Phase 3)

React · multipart/form-data
AI: photo vs spec check
not built yet

FIREBASE BaaS (Spark plan — no server billing)

Firestore (NoSQL · Document DB)

```
/projects/{id} · name, status, budget
/documents/{id} · file, category
/activityLog · user, action, ts
/users/{uid} · role, name
```

Firestore Auth (JWT · OAuth)

Google OAuth 2.0
idToken → Firestore rules
currentUser.uid scopes data
onAuthStateChanged()

FASTAPI (Python 3.11 · Uvicorn · Docker · localhost:8000)

REST API Routes

```
POST /api/upload/catalog xlsx
POST /api/upload/items xlsx
POST /api/match/run JSON
GET /api/items · POST /api/export
```

SQLite (SQL · file-based · matcher.db)

```
catalog_products id, code, name, price
internal_items id, code, name, qty
match_results item_id, product_id,
confidence_score, is_selected flag
```

EXPORT / FILE GENERATION

openpyxl (server-side)

```
Python · in FastAPI process
Styler.xlsx · navy header
Confidence % green/amber/red
POST /api/export → blob
```

JSZip (client-side · CDN)

Runs in browser — no server
OOXML parts → zip in RAM
generateAsync({type:'blob'})
anchor.click() → download

MATCHING PIPELINE (Python · in-process · 3 stages)

① TF-IDF

```
scikit-learn
word ngram (1,2)
char ngram (3,5)
cosine sim → top-k
```

② rapidfuzz

DeterministicFilter
fuzz.ratio() on code
exact → score 0.95
fuzzy >80% → +0.2

③ Heuristic Ranker

- top-k select & rank
- SQLite match_results
- is_selected flag
- LLM slot (v2 — future)

LEGEND

- NoSQL (Firestore) — projects, documents, activityLog, users

- SQL (SQLite) — catalog_products, internal_items, match_results

- — — HTTP REST (fetch · JSON · multipart/form-data)

— — — — Firebase SDK (WebSocket · onSnapshot realtime)

- Internal call (in-process Python / pipeline)

- SQLAlchemy ORM (FastAPI → SQLite)

TECH STACK SUMMARY

Frontend: Vanilla JS (dashboard) · React/Vite (matcher)

Backend: FastAPI · Python 3.11 · Uvicorn · Docker

DB: SQLite (matching) · Firestore NoSQL (projects)

ML: [scikit-learn](#) [TF-IDF](#) · [rapidfuzz](#) · [openpyxl](#)

Hosting: [Firebase Hosting \(SPA\)](#) · [Docker local \(API\)](#)